

Welcome to CSC331 Fall 2025 Data Structures

Professor Kevin Byron (kbyron@bmcc.cuny.edu)

Department of CIS, Room F-1030-I

Office hours: Mon 2pm-5pm (in-person and Zoom)

Office phone: 212-220-1490

Borough of Manhattan Community College/CUNY

CSC331 Fall 2025

Data Structures

Week 8:

Sample C++ code for graph evaluation
C++ file I-O, command line parameters
Sorting & searching algorithms
Recursion

Professor Kevin Byron
Department of CIS
Borough of Manhattan Community College/CUNY

Reminders

- Follow Monday schedule on: Fri 10-24-2025
- Program 3 due date
 - 1159pm Fri 10-31-2025
 - late submission receives no credit
- Exam 2
 - Section 501 Tue 11-11-2025 – Zoom 630pm-730pm
 - Section 172 Wed 11-12-2025 – In person room M-1001 630pm-730pm
 - Closed book, closed notes, no devices
 - Section 500 Wed 11-12-2025 – In person room N-453 1010am-1110am
 - Closed book, closed notes, no devices
- Program 4 due date
 - 1159pm Fri 11-21-2025
 - late submission receives no credit
- Posted on Brightspace
 - Week 8 lecture slides , graph evaluator sample code (chap12a.cpp), file I-O and command line parameters sample code (chap10a.cpp, chap10b.cpp, chap10c.cpp)

Graphs

- Sample definition, initialization and traversal of adjacency matrix for directed graph:

```
#include<iostream>
using namespace std;
int main(){
    int a[5][5] = {
        {0,0,0,0,0},
        {1,0,1,0,0},
        {0,0,0,1,0},
        {1,1,0,0,1},
        {0,0,1,0,0}
    };
    cout << "Matrix a follows: " << endl;
    for(int i=0; i<5; i++){
        for(int j=0; j<5; j++)
            cout << a[i][j] << " ";
        cout << endl;
    }
    return 0;
}
```

Graphs

- Sample definition, initialization and traversal of adjacency matrix for undirected, weighted graph:

```
#include<iostream>
using namespace std;
int main(){
    int b[7][7] = {
        {0,3,5,0,0,2,0},
        {3,0,0,8,0,6,0},
        {5,0,0,7,1,0,0},
        {0,8,7,0,4,0,0},
        {0,0,1,4,0,0,9},
        {2,6,0,0,0,0,10},
        {0,0,0,0,9,10,0}
    };
    cout << "Matrix b follows: " << endl;
    for(int i=0; i<7; i++){
        for(int j=0; j<7; j++)
            cout << b[i][j] << " ";
        cout << endl;
    }
    return 0;
}
```

Graphs

- Sample graph evaluation:

```
// chap12a.cpp
// kbyron@bmcc.cuny.edu
// 4/17/19
// C++ program to determine size of a graph from edges in a text file
// (1) vertices are integers in the range 0-99
// (2) one edge is represented on one input line as two vertices separated by one space
#include <fstream>
#include <iostream>
#include <string>
using namespace std;
int main () {
```

Graphs

- Sample graph evaluation:

```
int main () {
    bool V[100]; // set of vertices V
    string line,vs1,vs2;
    int pos1,v1,v2,vCount=0;
    ifstream myfile ("graph1.txt");
    for (int i = 0; i < 100; i++){
        V[i]=0;
    }
    if (myfile.is_open()){
        while ( getline (myfile,line) ){
            if(line.compare("end-of-file")){
                pos1 = line.find(' ');
                vs1 = line.substr(0, pos1);
                v1 = stoi(vs1);
                V[v1]=1;
                vs2 = line.substr(pos1 + 1);
                v2 = stoi(vs2);
                V[v2]=1;
            }
        }
        myfile.close();
    }
    else
        cout << "Unable to open file"<< endl;
    cout << "Graph vertices are: ";
```

Graphs

- Sample graph evaluation:

```
    cout << "Unable to open file"<< endl;
cout << "Graph vertices are: ";
for (int i = 0; i < 100; i++){
    if(V[i]){
        if(vCount)
            cout << ", ";
        else
            cout << " ";
        vCount++;
        cout << i;
    }
}
cout << "." << endl << "Graph size is " << vCount << "." << endl;
return 0;
}
```


Graphs

- Sample graph evaluation:

```
C:\Users\Kevin Byron\work\cpp\fy19>type graph1.txt
```

```
1 3
```

```
4 6
```

```
6 2
```

```
18 22
```

```
1 4
```

```
1 5
```

```
1 2
```

```
5 3
```

```
3 4
```

```
76 3
```

```
22 5
```

```
22 76
```

```
end-of-file
```

```
C:\Users\Kevin Byron\work\cpp\fy19>.\chap12a
```

```
Graph vertices are: 1, 2, 3, 4, 5, 6, 18, 22, 76.
```

```
Graph size is 9.
```

```
C:\Users\Kevin Byron\work\cpp\fy19>
```

Command line arguments and file I-O

```
// chap10b.cpp
// kbyron@bmcc.cuny.edu
// 4/14/19
// C++ program to demonstrate creating a text file
#include <iostream>
#include <fstream>
using namespace std;
int main () {
    ofstream myfile ("example.txt");
    if (myfile.is_open()){
        myfile << "line 1.\n";
        myfile << "line 2.\n";
        myfile << "line 3.\n";
        myfile.close();
        cout << "Text file created: example.txt" << endl;
    }
    else
        cout << "Unable to open file";
    return 0;
}
```

Command line arguments and file I-O

```
// chap10c.cpp
// kbyron@bmcc.cuny.edu
// 4/14/19
// C++ program to demonstrate reading a text file
#include <fstream>
#include <iostream>
#include <string>
using namespace std;
int main () {
    string line;
    ifstream myfile ("example.txt");
    if (myfile.is_open()){
        while ( getline (myfile,line) ){
            cout << line << '\n';
        }
        myfile.close();
    }
    else
        cout << "Unable to open file";
    return 0;
}
```

Command line arguments and file I-O

```
// chap10a.cpp
// kbyron@bmcc.cuny.edu
// 4/14/19
// C++ program to demonstrate command line arguments
#include <iostream>
using namespace std;
int main(int argc, char** argv) {
    cout << "You have entered " << argc << " arguments:" << "\n";
    for (int i = 0; i < argc; ++i)
        cout << argv[i] << "\n";
    return 0;
}
```

```
C:\Users\Kevin Byron\work\cpp\fy19>.\chap10a file1 file2
You have entered 3 arguments:
.\chap10a
file1
file2
```

```
C:\Users\Kevin Byron\work\cpp\fy19>_
```

Searching & sorting

The algorithm below will search for an element x in List L and will return “FOUND” if x is in the list and “NOT FOUND” otherwise. L has n items and $L[i]$ refers to the i^{th} element in L .

Linear Search Algorithm

- ① repeat as i varies from 1 to n
- ② if $L[i] = x$ then return “FOUND” and stop
- ③ return “NOT FOUND”

Note: The repeat loop spans lines 1 and 2.

Searching & sorting

Binary Search Algorithm

- ① Initialize $min \leftarrow 1$ and $max \leftarrow n$
- ② Repeat until $min > max$
- ③ $midpoint = \frac{1}{2}(min + max)$
- ④ compare x to $L[midpoint]$
 - (a) if $x = L[midpoint]$ then return “FOUND”
 - (b) if $x > L[midpoint]$ then $min \leftarrow midpoint + 1$
 - (c) if $x < L[midpoint]$ then $max \leftarrow midpoint - 1$
- ⑤ return “NOT FOUND”

Note: the repeat loop spans lines 2–4.

Searching & sorting

MergeSort Algorithm

function mergesort(L)

- ① if L has one element then return(L); otherwise continue
- ② $l_1 \leftarrow \text{mergesort}(\text{left half of } L)$
- ③ $l_2 \leftarrow \text{mergesort}(\text{right half of } L)$
- ④ $L \leftarrow \text{merge}(l_1, l_2)$
- ⑤ return(L)

Recursion

- The process of solving a problem by reducing it to smaller versions of itself is called recursion
- Recursion is a very powerful way to solve certain problems for which the solution would otherwise be more complicated
- For example, the factorial of a number can be computed using a recursive approach

Recursion

the factorial of 5, written $5!$, is $5 \times 4 \times 3 \times 2 \times 1 = 120$

$$4! = 4 \times 3 \times 2 \times 1 = 24$$

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 5 \times (4 \times 3 \times 2 \times 1) = 5 \times 4!$$

$$n! = n \times (n - 1)! \quad \text{if } n > 0$$

$$0! = 1$$

Recursion

Recursive definition: A definition in which something is defined in terms of a smaller version of itself.

From the previous example (factorial), it is clear that:

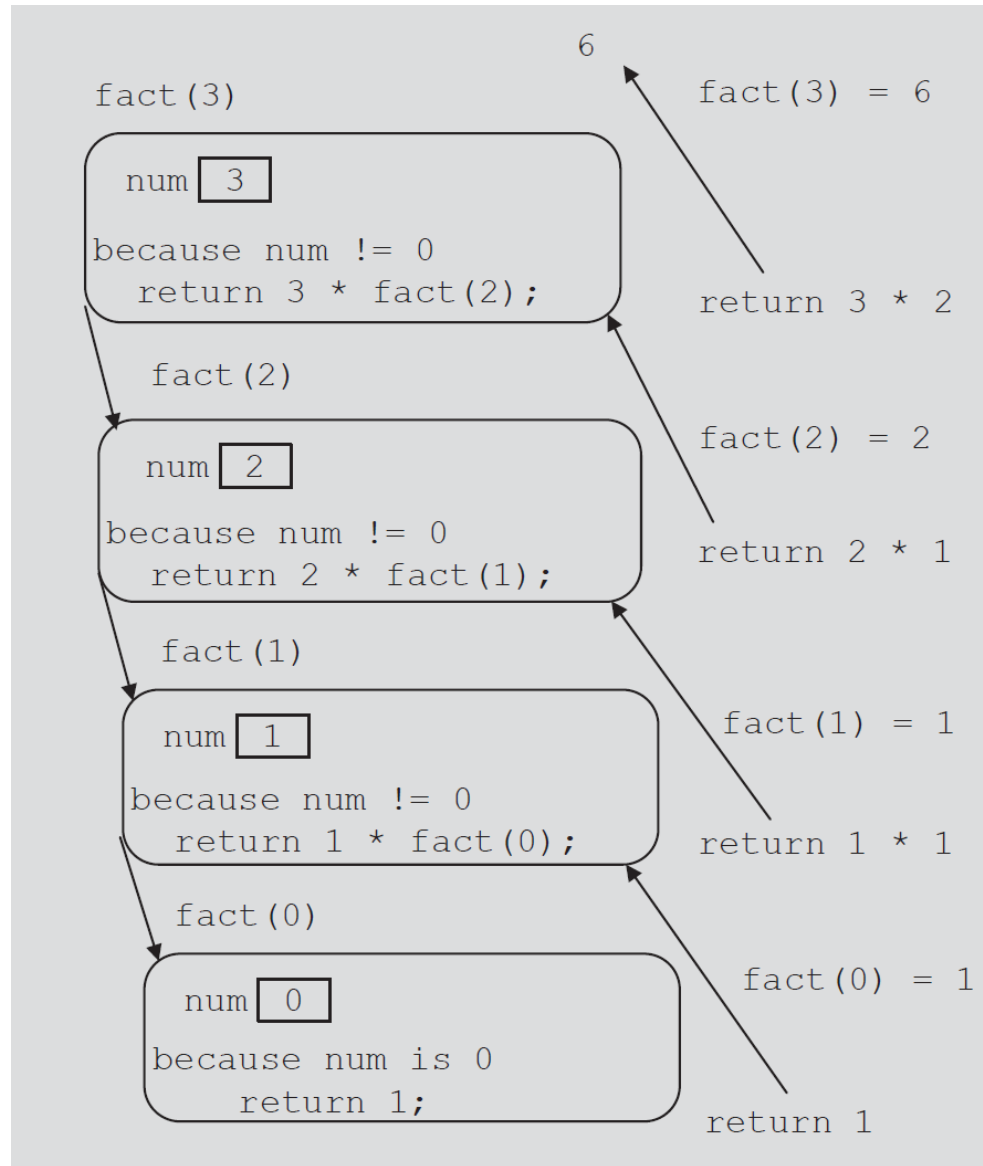
1. Every recursive definition must have one (or more) base cases.
2. The general case must eventually be reduced to a base case.
3. The base case stops the recursion.

```
int fact(int num)
{
    if (num == 0)
        return 1;
    else
        return num * fact(num - 1);
}
```

```
cout << fact(3) << endl;
```

Recursion

- Execution of fact(3):



Recursion

Consider the following recursive function:

```
void exercise(int x)
{
    if (x > 0 && x < 10)
    {
        cout << x << " ";
        exercise(x + 1);
    }
}
```

What is the output of the following statements?

- a. `exercise(0);`
- b. `exercise(5);`
- c. `exercise(10);`
- d. `exercise(-5);`

Recursion

Consider the following function:

```
int test(int x, int y)
{
    if (x == y)
        return x;
    else if (x > y)
        return (x + y);
    else
        return test(x + 1, y - 1);
}
```

What is the output of the following statements?

- a. `cout << test(5, 10) << endl;`
- b. `cout << test(3, 9) << endl;`

Recursion

Consider the following function:

```
int func(int x)
{
    if (x == 0)
        return 2;
    else if (x == 1)
        return 3;
    else
        return (func(x - 1) + func(x - 2));
}
```

What is the output of the following statements?

- a. `cout << func(0) << endl;`
- b. `cout << func(1) << endl;`
- c. `cout << func(2) << endl;`
- d. `cout << func(5) << endl;`

Recursion

(Greatest Common Divisor) Given two integers x and y , the following recursive definition determines the greatest common divisor of x and y , written $\text{gcd}(x, y)$:

$$\text{gcd}(x, y) = \begin{cases} x & \text{if } y = 0 \\ \text{gcd}(y, x \% y) & \text{if } y \neq 0 \end{cases}$$

Note: In this definition, $\%$ is the mod operator.

Write a recursive function, `gcd`, that takes as parameters two integers and returns the greatest common divisor of the numbers. Also, write a program to test your function.

Recursion

Write a recursive function, `power`, that takes as parameters two integers x and y such that x is nonzero and returns x^y . You can use the following recursive definition to calculate x^y . If $y \geq 0$,

$$power(x, y) = \begin{cases} 1 & \text{if } y = 0 \\ x & \text{if } y = 1 \\ x \times power(x, y - 1) & \text{if } y > 1. \end{cases}$$

If $y < 0$,

$$power(x, y) = \frac{1}{power(x, -y)}.$$

Also, write a program to test your function.

Searching & sorting

How would mergesort sort the list (9 2 8 4 1 3) into increasing order?

To help show what is going on, the sorted lists that are about to be merged are shown in bold.

9 2 8 4 1 3 → 9 2 8 4 1 3 → 9 2 8 4 1 3

9 2 8 4 1 3 → **2 9** 8 4 1 3 → 2 8 9 4 1 3

2 8 9 4 1 3 → 2 8 9 **4 1** 3 → 2 8 9 4 **1** 3

2 8 9 **1 4** 3 → 2 8 9 **1 3 4** → 1 2 3 4 8 9

Mergesort operation

- Given the following values to be sorted using recursive mergesort:
9,2,8,4,1,3
- These values are passed on each call of the mergesort function and returned from each call of the merge function:
 - MS1: 9,2,8,4,1,3
 - MS2: 9,2,8
 - MS3: 9,2
 - MS4: 9
 - MS5: 2
 - MG1: (9)+(2)->2,9
 - MS6: 8
 - MG2: (2,9)+(8)->2,8,9
 - MS7: 4,1,3
 - MS8: 4,1
 - MS9: 4
 - MS10: 1
 - MG3: (1)+(4)->1,4
 - MS11: 3
 - MG4: (1,4)+(3)->1,3,4
 - MG5: (2,8,9)+(1,3,4)->1,2,3,4,8,9

Mergesort operation - practice

- Given the following values to be sorted using recursive mergesort:
'Sha', 'Lee', 'Abu', 'Luk' , 'Zhe', 'Jea', 'Mam', 'Wei'
 1. Show the values passed on the 2nd call of the mergesort function.
 2. Show the values passed on the 4th call of the mergesort function.
 3. Show the result returned from the 1st call of the merge function.
 4. Show the result returned from the 4th call of the merge function.

Mergesort operation - solutions

- Given the following values to be sorted using recursive mergesort:
'Sha', 'Lee', 'Abu', 'Luk' , 'Zhe', 'Jea', 'Mam', 'Wei'
 1. Show the values passed on the 2nd call of the mergesort function.
MS2: 'Sha', 'Lee', 'Abu', 'Luk'
 2. Show the values passed on the 4th call of the mergesort function.
MS4: 'Sha'
 3. Show the result returned from the 1st call of the merge function.
MG1: ('Sha') + ('Lee') -> ('Lee', 'Sha')
 4. Show the result returned from the 4th call of the merge function.
MG4: ('Zhe') + ('Jea') -> ('Jea', 'Zhe')

Assignment

- Read Malik chapter 12 - graphs
- Read Malik chapter 6 - recursion
- Read Malik chapter 9 - searching
- Read Malik chapter 10 - sorting